



Oracle SQL Course – Brief Overview



Course Goal:

To introduce learners to **Oracle SQL**, enabling them to write queries, manipulate data, and understand Oracle-specific database features.



Key Topics:

1. Introduction to Oracle SQL

- Understanding Oracle Database
- SQL vs PL/SQL
- Using Oracle SQL Developer

2. Basic SQL Operations

- **SELECT, FROM, WHERE, ORDER BY**
- Filtering and sorting data

3. Functions & Operators

- Aggregate functions: **COUNT, SUM, AVG**
- String, number, and date functions

4. Data Manipulation (DML)

- **INSERT, UPDATE, DELETE**

- Using **COMMIT** and **ROLLBACK**

5. Data Definition (DDL)

- **CREATE, ALTER, DROP** tables
- Constraints: **PRIMARY KEY, UNIQUE, FOREIGN KEY**

6. Joins & Subqueries

- **INNER, LEFT, RIGHT** joins
- Subqueries within **SELECT, WHERE, FROM**

7. Oracle-Specific Features

- **DUAL** table
- **ROWNUM, SYSDATE**
- Sequences for auto-incrementing values

Outcome:

By the end, learners will be able to:

- Query and manage data in Oracle databases
- Understand and use Oracle-specific SQL features
- Write optimized, real-world SQL queries



Introduction to SQL (Structured Query Language)



1. Database

- A **database** is a **structured collection of interrelated data**.
- It stores data in an organized way (tables, rows, columns).
- Example: A database might store customer details, sales data, or employee records.



Think of a **database** as a **warehouse** where all the data is stored.



2. DBMS (Database Management System)

- A **DBMS** is **software** that allows you to **create, manage, and interact with a database**.
- It handles data storage, retrieval, security, backups, and concurrency.
- Examples: Oracle, MySQL, SQL Server, PostgreSQL



Think of a **DBMS** as the **manager** of the warehouse (database), handling everything inside.



3. SQL (Structured Query Language)

- **SQL** is the **language** used to **communicate with the DBMS**.

- It allows you to perform actions like:
 - Create tables (**CREATE**)
 - Insert data (**INSERT**)
 - Query data (**SELECT**)
 - Update or delete data (**UPDATE, DELETE**)

👉 Think of **SQL** as the **instructions or commands** you give to the DBMS manager to get work done in the warehouse.



Types of DBMS

DBMSs are categorized based on how they store and manage data. There are four main types:

1. Object-Oriented DBMS (OODBMS)

- Structure: Based on object-oriented programming (OOP).
- Characteristics:
 - Data stored as objects (with attributes and methods).
 - Supports complex data types.
- Examples: ObjectDB, db4o



Think of: Storing entire objects like in Java or C++

2. Hierarchical DBMS

- **Structure:** Tree-like structure (parent-child relationship).
- **Characteristics:**
 - One parent, many children.
 - Fast and simple if data follows a fixed hierarchy.
- **Example:** IBM IMS (used in banking, airline systems)

 **Think of:** A folder system in your computer.

3. Network DBMS

- **Structure:** Graph structure (many-to-many relationships).
- **Characteristics:**
 - A child can have multiple parents.
 - More flexible than hierarchical DBMS.
- **Example:** IDMS (used in telecom and logistics)

 **Think of:** A social network where one person can be connected to many people.

4. Relational DBMS (RDBMS)

- **Structure:** Table format (rows and columns).
- **Characteristics:**
 - Uses **SQL** (Structured Query Language).

- Most widely used DBMS type.

- **Examples:** MySQL, Oracle, SQL Server, PostgreSQL

🧠 **Think of:** Excel spreadsheets that can be linked together.



What is Metadata?

Metadata is data that describes other data. It provides information about a data resource, such as its structure, content, origin, time, or format.



Simple Definition:

"Metadata is data **about** data."



Real-World Examples of Metadata

Data (Main Content)

Metadata (Describes the Data)



A Word Document

File name, author, word count, creation date, file size



A Photo

Camera model, resolution, date taken, GPS location, file format



A Song in Spotify

Song title, artist, album, genre, duration

 A YouTube Video	Title, uploader, tags, resolution, duration, upload date
 A Table in a Database	Table name, column names, data types, primary key, row count
 A GIS Map File	Location coordinates, projection type, scale, creator

Example: Metadata in a Database Table

Let's say we have a table called **Employees**.

Actual Data:

ID	Name	Department	Salary
1	John Doe	Sales	50000

Metadata:

- Table name: **Employees**
- Columns: **ID, Name, Department, Salary**
- Data types: **ID (INTEGER), Name (VARCHAR)**, etc.
- Constraints: **ID** is PRIMARY KEY

- Created on: 2024-01-01
- Total Rows: 100

📌 This metadata tells **how the data is stored and how it can be used**, but not the data itself.

✓ How to Install Oracle Database (Step-by-Step Guide)

Here's a **beginner-friendly guide** to installing **Oracle Database** on your system. This covers the **Oracle Database 21c or 19c Express Edition (XE)**, which is free and ideal for learning purposes.

🧰 System Requirements (Oracle XE)

- **OS:** Windows 10/11 (64-bit) or Linux
- **RAM:** Minimum 2 GB (4 GB recommended)
- **Disk Space:** At least 10 GB free
- **Privileges:** Admin rights

📄 Step 1: Download Oracle Database

1. Visit the official Oracle download page:
<https://www.oracle.com/database/technologies/xe-downloads.html>
2. Choose the appropriate version:
 - **Oracle 21c XE** (latest)
 - Or **Oracle 19c XE** (if you want a long-term support version)

3. Click "**Download**", accept the license agreement, and sign in or create an Oracle account.
-

Step 2: Install Oracle Database (Windows Example)

1. Run the Installer:

- Locate the downloaded **.exe** file and double-click to start.

2. Follow Setup Wizard:

- Choose an installation folder.
- Set a **password** for the SYS, SYSTEM, and PDBADMIN accounts.
- Let the installer run and complete the process.

3. Installation Complete

- Oracle Database will be installed along with:
 - Oracle Database Server
 - Oracle Listener
 - Oracle SQL*Plus
 - Oracle SQL Developer (GUI for working with SQL)
-

Step 3: Verify and Login

To connect using SQL*Plus:

Open Command Prompt and enter:

```
bash
```

```
sqlplus sys as sysdba
```

Then enter the password you set during installation.

To use Oracle SQL Developer:

1. Open Oracle SQL Developer.
2. Create a new connection with:
 - **Username:** `system`
 - **Password:** [your password]
 - **Hostname:** `localhost`
 - **Port:** `1521`
 - **Service Name:** `XEPDB1` (or `XE` for older versions)

Click **Test** → then **Connect**.

You're Ready!

Now you can start creating tables, writing queries, and learning Oracle SQL!



Sub-Languages in SQL

SQL (Structured Query Language) is divided into **five main sub-languages**, each serving a specific function in interacting with the database.

Sub-Language	Purpose	Key Commands	Example
DQL (Data Query Language)	Retrieve data	SELECT	SELECT * FROM employees;
DML (Data Manipulation Language)	Manipulate data	INSERT, UPDATE, DELETE	INSERT INTO dept VALUES (1, 'HR');
DDL (Data Definition Language)	Define/change structure	CREATE, ALTER, DROP, TRUNCATE	CREATE TABLE dept (...);
DCL (Data Control Language)	Control access	GRANT, REVOKE	GRANT SELECT ON employees TO user1;
TCL (Transaction Control Language)	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT	COMMIT;

Datatypes:

What Are Datatypes in Oracle?

In Oracle Database, a datatype defines the kind of data a column, variable, or expression can hold. It tells the database how much space to allocate and how to store and manipulate the data.

Why Are Datatypes Important?

- They ensure **data integrity** (e.g., storing only numbers in a salary column).
- They improve **performance** by optimizing how data is stored.
- They help Oracle enforce rules like **constraints, sorting, and indexing**.

Categories of Datatypes in Oracle:

1. **Numeric Datatypes** - Store numbers.
 2. **Character Datatypes** - Store text or characters.
 3. **Date and Time Datatypes** - Store dates and times.
 4. **Large Object (LOB) Datatypes** - Store large data like files or documents.
 5. **Long and raw datatype** - Stores **very large text data** (like long descriptions or articles).
 6. **ROWID/UROWID** - Store row addresses.
-

Numeric Datatypes

Oracle provides several **numeric datatypes** to store numbers—both integers and floating-point numbers. Here's a breakdown:

1. **NUMBER**

The most commonly used numeric datatype.

sql

NUMBER(p, s)

- **p** = precision (total digits)
- **s** = scale (digits after the decimal point)

Example	Description
NUMBER	A number with up to 38 digits of precision
NUMBER(6)	A number with up to 6 digits, no decimal
NUMBER(8,2)	6 digits before the decimal, 2 after

NUMBER(*,0) Any number with 0 digits
after decimal (integer)

✓ Used for both integers and decimals.

✓ 2. INTEGER / INT / SMALLINT

- These are just different names for **whole numbers**.
- All are the same as **NUMBER** without decimal part.

Example:

sql

INTEGER -- Same as NUMBER(38)

✓ 3. FLOAT

Stores approximate numeric values.

sql

FLOAT(p)

- **p** = binary precision (not decimal digits)

Example	Description
<code>FLOAT</code>	Approximate number, default precision 126 bits
<code>FLOAT(10)</code>	Approximate with binary precision of 10 bits

 If you want precise decimal values, use `NUMBER` instead of `FLOAT`.

4. `BINARY_FLOAT` and `BINARY_DOUBLE`

High-performance **floating-point types** using IEEE 754.

Datatype	Precision	Description
<code>BINARY_FLOAT</code>	32-bit	Fast, approximate floating-point number
<code>BINARY_DOUBLE</code>	64-bit	Double-precision floating-point number

✓ Use these for performance-critical calculations where approximate values are acceptable.

Character Datatypes

These are used to store text – like names, addresses, or messages.

1. CHAR(n)

- Stores fixed-length text.
- If the text is shorter, it fills with spaces.

Example:

```
sql
CHAR(5)
-- 'Hi' is stored as 'Hi   '
```

2. VARCHAR2(n)

- Stores variable-length text.
- Only stores what you type (no extra spaces).

Example:

```
sql
VARCHAR2(5)
-- 'Hi' is stored as 'Hi'
```

 This is the most commonly used text type.

✓ 3. NCHAR(n)

- Like **CHAR**, but supports Unicode (multi-language).
-

✓ 4. NVARCHAR2(n)

- Like **VARCHAR2**, but supports Unicode.

Quick Comparison Table:

Datatype	Fixed or Variable	Used For
CHAR(n)	Fixed	Short, fixed-length text
VARCHAR2(n)	Variable	Regular text (most common)
NCHAR(n)	Fixed	Unicode fixed text
NVARCHAR2(n)	Variable	Unicode variable text

Date and Time Datatypes

These are used to store dates and times, like birth dates, timestamps, or event times.

✓ 1. DATE

- Stores date and time (year, month, day, hour, minute, second).
- Default format: **DD-MON-YY** or **DD-MON-YYYY**.

Example :

sql

DATE

```
-- '08-JUL-2025 10:30:00'
```

- Most commonly used for storing date and time.
-

✓ 2. TIMESTAMP

- Like **DATE** but also stores fractions of a second.
- **Example:**

```
sql
```

```
TIMESTAMP
```

```
-- '08-JUL-2025 10:30:00.123456'
```

✓ 3. INTERVAL YEAR TO MONTH

- Stores a period of time in years and months.

Example:

```
sql
```

```
INTERVAL '2-6' YEAR TO MONTH
```

```
-- 2 years and 6 months
```

✓ 4. INTERVAL DAY TO SECOND

- Stores a duration in days, hours, minutes, seconds (and fractions).

Example:

```
sql
INTERVAL '3 04:30:15' DAY TO SECOND
-- 3 days, 4 hours, 30 minutes, 15 seconds
```

 Quick Summary Table:

Datatype	Stores
DATE	Date + time (no fractional seconds)
TIMESTAMP	Date + time + fractional seconds
INTERVAL YEAR TO MONTH	Time period in years and months
INTERVAL DAY TO SECOND	Time duration in days, hours, seconds

 Large Object (LOB) Datatypes

LOB (Large Object) datatypes in Oracle are used to store large amounts of data, such as text, images, audio, video, and other files.

There are two types of LOBs:

- Internal LOBs: Stored in the database (e.g., CLOB, BLOB)
- External LOBs: Stored outside the database (e.g., BFILE)

 Types of LOB Datatypes

Datatype	Type	Stores	Max Size	Stored Where?
----------	------	--------	----------	---------------

CLOB	Character	Large text data	4 GB+	Inside database
NCLOB	Character	Large Unicode text	4 GB+	Inside database
BLOB	Binary	Large binary data	4 GB+	Inside database
BFILE	Binary	Large binary files	4 GB+ (read-only)	Outside database (OS file)

Explanation of Each:

1. CLOB (Character Large Object)

- Stores **large plain text** like documents, articles, logs.
- Uses **CHARACTER set** of the database.

2. NCLOB (National Character Large Object)

- Like CLOB, but supports **multilingual Unicode text** (NCHAR data).

3. BLOB (Binary Large Object)

- Stores **binary files**, such as images, videos, PDFs.
- Data is stored **exactly as-is**, no character conversion.

4. BFILE (Binary File)

- Stores a **pointer to an external file** (not stored in the DB).
- **Read-only** from PL/SQL.
- File must be in a directory that Oracle has access to.

◆ Long and raw datatype

1. LONG Datatype

- **Purpose:** Stores **very large text data** (like long descriptions or articles).
- **Type:** Character (Text)
- **Maximum Size:** Up to 2 GB
- **Limitations:**
 - Only one **LONG** column per table.
 - Cannot use in **WHERE, GROUP BY, ORDER BY, JOIN**, etc.
- **Status:**  **Deprecated** (Oracle recommends using **CLOB** instead).

Example:

```
sql

CREATE TABLE books (

    book_id NUMBER,

    summary LONG

);
```

2. RAW Datatype

- **Purpose:** Stores binary data, not readable as text (like images, encrypted data).

- **Type:** Binary
- **Maximum Size:** Up to 2000 bytes
maximum size is 32767 bytes if max_string_size_extended
- **Usage:** Stores data "as-is", with no character set conversion.

Example:

sql

```
CREATE TABLE files (  
  
    file_id NUMBER,  
  
    file_data RAW(1000)  
  
);
```

3.LONG RAW (Related to RAW)

- Same as **RAW**, but supports larger binary data (up to 2 GB).
- Also deprecated – use **BLOB** instead.

Summary Table

Datatype	Type	Stores	Max Size	Status	Modern Alternative
----------	------	--------	-------------	--------	-----------------------

LONG	Character	Large text	2 GB	Deprecated	CLOB
RAW(n)	Binary	Small binary data	2000 B	Still used	—
LONG RAW	Binary	Large binary data	2 GB	Deprecated	BLOB

◆ Rowid and urowid datatype

1. ROWID (Row Identifier)

- **Type:** Pseudocolumn / Datatype
- **Purpose:** Uniquely identifies the **physical address** of a row in a table.
- **Contents:** Includes data like:
 - Block number
 - File number
 - Row position in block

✓ Key Points:

- Each row in a table has a unique ROWID.
- Very fast for accessing rows.

- Often used for **performance optimization** and **row-level updates**.

2. UROWID (Universal ROWID)

- **Type:** Universal Row Identifier
- **Purpose:** Can store both **physical** and **logical** row addresses.
- Used especially for **tables with object types** or **index-organized tables (IOTs)**.
- More flexible than **ROWID**.

Why UROWID?

- **ROWID** only works for regular heap-organized tables.
- **UROWID** can also handle rows that **don't have a fixed physical location**, such as:
 - Rows in IOTs
 - Rows in foreign tables (like through **DBMS_LOB**)

♦ What is DDL

In SQL (Structured Query Language), DDL stands for Data Definition Language. It's one of the key sublanguages of SQL, used to define and manage the structure of database objects such as tables, indexes, schemas, and constraints.

What DDL Does:

DDL commands create, modify, and delete database structures—but not the data stored in them.

✓ Common DDL Commands :

Command	Description
CREATE	Creates new database objects (e.g., tables, views, schemas).
ALTER	Modifies existing database objects (e.g., add/remove columns).
DROP	Deletes existing database objects.
Flashback	Used to recover data from a previous point in time without restoring from backup.
purge	Permanently removes objects from the database Recycle Bin
TRUNCATE	Removes all rows from a table quickly, without logging individual row deletions.
RENAME	Changes the name of a database object (supported in some RDBMSs).

CREATE

- Purpose: Creates a new database object, such as a table, schema, or index.

***Syntax:**

sql

```
CREATE TABLE table_name (  
  
    column1 datatype [constraints],  
  
    column2 datatype [constraints],  
  
    ...  
  
);
```

***Example:**

sql

```
CREATE TABLE Students (  
  
    ID INT PRIMARY KEY,  
  
    Name VARCHAR(50) ,
```

```
Age INT
```

```
);
```

Notes: Defines the structure (columns, datatypes, constraints) of a new table or other object. Fails if the object already exists unless specified otherwise (e.g., IF NOT EXISTS).

ALTER

- Purpose: Modifies an existing database object, such as adding, modifying, or dropping columns in a table.

***Syntax:**

```
sql
```

```
ALTER TABLE table_name
```

```
[ADD | MODIFY | DROP ] column_name datatype  
[constraints];
```

EXAMPLES:

***Add a column:**

```
sql
```

```
ALTER TABLE Students ADD Email VARCHAR(100);
```

***Modify a column's datatype:**

sql

```
ALTER TABLE Students MODIFY Age VARCHAR(3);
```

***Drop a column:**

sql

```
ALTER TABLE Students DROP COLUMN Email;
```

***Rename a column:**

sql

```
ALTER TABLE table_name
```

```
Rename column old_column_name to new column name;
```

TRUNCATE

- Purpose: Removes all rows from a table, resetting it to an empty state while preserving its structure (columns, constraints, indexes).
- Syntax:

sql

```
TRUNCATE TABLE table_name;
```

*Example:

sql

```
TRUNCATE TABLE Students;
```

RENAME

- Purpose: Changes the name of an existing database object, such as a table or column.
- Syntax (varies by database):

*For tables:

sql

```
ALTER TABLE old_table_name RENAME TO  
new_table_name;
```

or (in some databases like Oracle):

sql

```
RENAME old_table_name TO new_table_name;
```

DROP

- Definition: Removes a database object (e.g., table, index, schema) from the database.
- Why DDL?: Modifies the database structure by deleting object metadata.

***Syntax:**

sql

```
DROP TABLE table_name;
```

***Example:**

sql

```
DROP TABLE Students;
```

Deletes the Students table, moving it to the recycle bin (in Oracle, if enabled).

FLASHBACK

- Definition: Restores a dropped object or reverts a table/database to a prior state (Oracle-specific).
- Why DDL?: Affects object metadata or structural state (e.g., recovering a table).

***Syntax:**

sql

```
FLASHBACK TABLE table_name TO BEFORE DROP;
```

or

sql

```
FLASHBACK TABLE table_name TO TIMESTAMP timestamp;
```

***EXAMPLE:**

```
sql
```

```
FLASHBACK TABLE Students TO BEFORE DROP;
```

Restores the Students table from the recycle bin.

PURGE

- Definition: Permanently deletes a dropped object or clears the recycle bin, preventing recovery (Oracle-specific).
- Why DDL?: Removes object metadata from the database.

***Syntax:**

```
sql
```

```
PURGE TABLE table_name;
```

or

```
sql
```

```
PURGE RECYCLEBIN;
```

***Example:**

sql

```
PURGE TABLE Students;
```

Permanently deletes the Students table from the recycle bin.

♦ What is DML

Data Manipulation Language (DML) in SQL refers to a set of commands used to manipulate data within a database's tables. Unlike Data Definition Language (DDL), which manages database structure (e.g., tables, schemas), DML focuses on inserting, updating, deleting, and retrieving data in tables.

- Common DML Commands:
 - INSERT : Add new data.
 - UPDATE : Modify existing data.
 - DELETE : Remove data.
 - INSERT ALL : Inserts multiple rows into one or more tables in a single statement, optionally with conditions.
 - MERGE : Conditionally updates, inserts, or deletes rows in a target table based on a comparison with a source table or dataset.

INSERT

INSERT is a DML command that adds one or more rows of data to a specified table in a database.

Syntax

sql

```
INSERT INTO table_name (column1, column2, ..., columnN)
VALUES (value1, value2, ..., valueN);
```

✓ Example:

Suppose you have a table called employees:

```
sql
CREATE TABLE employees (
    employee_id NUMBER,
    first_name VARCHAR2(50),
    last_name VARCHAR2(50),
    hire_date DATE,
    salary NUMBER
);
```

You can insert a new employee like this:

```
sql
INSERT INTO employees (employee_id, first_name, last_name, hire_date,
salary)
VALUES (101, 'John', 'Doe', TO_DATE('2025-07-10', 'YYYY-MM-DD'),
50000);
```

Insert Without Column Names:

If you're inserting values into **all columns in the correct order**, you can omit the column list:

```
sql
INSERT INTO employees
VALUES (102, 'Jane', 'Smith', TO_DATE('2025-07-01', 'YYYY-MM-DD'),
60000);
```

✓ Basic Syntax with DATE:

```
sql
INSERT INTO table_name (date_column)
VALUES (TO_DATE('YYYY-MM-DD', 'YYYY-MM-DD'));
```

Example :

```
sql
INSERT INTO events (event_id, event_name, event_date)
VALUES (1, 'Meeting', TO_DATE('2025-07-10', 'YYYY-MM-DD'));
```

✓ Basic Syntax with TIMESTAMP:

```
sql
INSERT INTO table_name (timestamp_column)
VALUES (TO_TIMESTAMP('YYYY-MM-DD HH24:MI:SS.FF', 'YYYY-MM-DD
HH24:MI:SS.FF'));
```

Example :

```
sql
INSERT INTO events (event_id, event_name, event_time)
VALUES (2, 'System Upgrade', TO_TIMESTAMP('2025-07-10 14:30:00.123',
'YYYY-MM-DD HH24:MI:SS.FF'));
```

Update

In Oracle SQL, the **UPDATE** statement is a DML (Data Manipulation Language) command used to modify existing records in a table.

Syntax:

```
sql
UPDATE table_name
SET column1 = value1,
    column2 = value2,
    ...
WHERE condition;
```

- ♦ **Important:** Always use a WHERE clause to avoid updating all rows unless that is your intention.

Example:

```
sql
CREATE TABLE employees (
    employee_id NUMBER,
    first_name  VARCHAR2(50),
    last_name   VARCHAR2(50),
    salary      NUMBER,
    hire_date   DATE
);
```

Example 1: Update a Single Column

```
sql
UPDATE employees
SET salary = 60000
WHERE employee_id = 101;
```

This updates the salary of the employee with employee_id = 101.

Example 2: Update Multiple Columns

```
sql
```

```
UPDATE employees
SET first_name = 'John',
    last_name = 'Smith'
WHERE employee_id = 102;
```

Commit:

The COMMIT statement is used to **permanently save** those changes to the database.

Syntax:

```
sql
UPDATE table_name
SET column1 = value1,
    column2 = value2
WHERE condition;

COMMIT;
```

- COMMIT makes the changes permanent and visible to all users.
- If you do **not** issue COMMIT, the changes are only visible in your session and can be **rolled back** using ROLLBACK.

Delete

In SQL, the DELETE statement is a **DML (Data Manipulation Language)** command used to remove one or more rows from a table.

Basic DELETE Syntax:

sql

```
DELETE FROM table_name
```

```
WHERE condition;
```

 **Important:** Always use a WHERE clause unless you want to delete **all rows** from the table.

Example:

sql

```
CREATE TABLE employees (  
  
    employee_id NUMBER,  
  
    first_name  VARCHAR2(50),  
  
    last_name   VARCHAR2(50),  
  
    department  VARCHAR2(50)  
  
);
```

Example 1: Delete a Specific Row

sql

```
DELETE FROM employees
```

```
WHERE employee_id = 101;
```

Deletes the row where employee_id = 101.

Example 2: Delete Based on a Condition

sql

```
DELETE FROM employees
```

```
WHERE department = 'HR';
```

Deletes all employees in the HR department.

Example 3: Delete All Rows (but keep table structure)

```
sql
```

```
DELETE FROM employees;
```

 This deletes **all rows** from the table but keeps the table itself.

Example 4: Use with COMMIT

```
sql
```

```
DELETE FROM employees
```

```
WHERE department = 'IT';
```

```
COMMIT;
```

Makes the deletion permanent.

Insert all

In SQL, INSERT ALL is a **DML** statement that allows you to **insert multiple rows into one or more tables** using a single SQL statement. It is very useful when you want to insert multiple records efficiently.

Syntax of INSERT ALL:

sql

INSERT ALL

```
    INTO table_name (column1, column2, ...) VALUES (value1, value2, ...)
```

```
    INTO table_name (column1, column2, ...) VALUES (value1, value2, ...)
```

```
    ...
```

SELECT * FROM dual;

- ♦ DUAL is a special one-row, one-column table used in Oracle when a SELECT is required syntactically.

Example: INSERT ALL into One Table

sql

INSERT ALL

```
    INTO employees (employee_id, first_name, last_name, salary)
```

```
    VALUES (201, 'Alice', 'Smith', 60000)
```

```
    INTO employees (employee_id, first_name, last_name, salary)
```

```
    VALUES (202, 'Bob', 'Johnson', 62000)
```

```
    INTO employees (employee_id, first_name, last_name, salary)
```

```
    VALUES (203, 'Charlie', 'Williams', 58000)
```

```
SELECT * FROM dual;
```

✓ This inserts **three rows** into the employees table in one SQL command.

Insert into Multiple Tables (Advanced Usage)

```
sql
```

```
INSERT ALL
```

```
    INTO employees (employee_id, first_name, last_name, salary)
```

```
VALUES (204, 'Diana', 'Brown', 70000)
```

```
    INTO audit_log (log_id, action, action_date)
```

```
VALUES (1, 'Insert into employees', SYSDATE)
```

```
SELECT * FROM dual;
```

This example assumes you have a second table called audit_log.

◆ Key Notes

- INSERT ALL is **not the same** as BULK INSERT or INSERT INTO ... SELECT.
- Use INSERT FIRST if you want to insert into **only one** table based on conditions (first match only).
- You **must** use SELECT * FROM dual at the end, even if inserting hardcoded values.

Syntax: INSERT ALL with Subquery

sql

INSERT ALL

INTO table1 (col1, col2) VALUES (sub_col1, sub_col2)

INTO table2 (col3, col4) VALUES (sub_col3, sub_col4)

SELECT sub_col1, sub_col2, sub_col3, sub_col4

FROM source_table

WHERE condition;

♦ The subquery pulls data from a source table and the INSERT ALL pushes the same data (or different parts of it) into one or more destination tables.

Merge

In Oracle SQL, the MERGE statement is a DML command used to combine INSERT and UPDATE (or DELETE) in a single operation.

It's also called "UPSERT", because it will insert new rows or update existing rows depending on whether a match is found.

Syntax of MERGE in Oracle:

sql

MERGE INTO target_table t

USING source_table s

ON (t.key_column = s.key_column)

WHEN MATCHED THEN

UPDATE SET t.col1 = s.col1, t.col2 = s.col2

WHEN NOT MATCHED THEN

```
INSERT (col1, col2, col3)

VALUES (s.col1, s.col2, s.col3);
```

Example Tables:

sql

-- Target table

```
CREATE TABLE employees (

    emp_id    NUMBER PRIMARY KEY,

    name      VARCHAR2(50),

    salary    NUMBER

);
```

-- Source table

```
CREATE TABLE new_employees (

    emp_id    NUMBER,

    name      VARCHAR2(50),

    salary    NUMBER

);
```

Example: MERGE to Update or Insert

sql

```
MERGE INTO employees e
USING new_employees n
ON (e.emp_id = n.emp_id)
WHEN MATCHED THEN
    UPDATE SET e.name = n.name,
               e.salary = n.salary
WHEN NOT MATCHED THEN
    INSERT (emp_id, name, salary)
    VALUES (n.emp_id, n.name, n.salary);
```



What this does:

- If emp_id exists in employees, it updates the name and salary.
- If emp_id does **not** exist, it inserts the row into employees.

♦ What is DRL or DQL

DRL (Data Retrieval Language) or **DQL (Data Query Language)** is a subset of SQL used **only to query data** from one or more tables in a database. In Oracle SQL, the **SELECT** statement is the **only command** in this category.

Purpose of DRL / DQL in Oracle

- Retrieve data without modifying it
- Analyze, view, or filter data
- Not transactional (does not need COMMIT or ROLLBACK)

✓ Main Command: SELECT

```
sql
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```



Example Table: employees

Let's say you have this table:

```
sql
CopyEdit
CREATE TABLE employees (
    emp_id      NUMBER,
    first_name   VARCHAR2(50),
    last_name    VARCHAR2(50),
    department   VARCHAR2(50),
    salary       NUMBER
);
```

With data:

```
sql
CopyEdit
```

```
INSERT INTO employees VALUES (101, 'Alice', 'Smith', 'HR', 50000);
INSERT INTO employees VALUES (102, 'Bob', 'Johnson', 'IT', 65000);
INSERT INTO employees VALUES (103, 'Charlie', 'Lee', 'IT', 62000);
INSERT INTO employees VALUES (104, 'Diana', 'White', 'Finance', 58000);
COMMIT;
```

DQL / DRL Examples in Oracle SQL

♦ 1. Retrieve All Data

sql

```
SELECT * FROM employees;
```

✓ Shows all records and columns from the table.

♦ 2. Select Specific Columns

sql

```
SELECT first_name, department FROM employees;
```

✓ Retrieves only names and departments.

♦ 3. Filter with WHERE Clause

sql

```
SELECT first_name, salary
FROM employees
WHERE department = 'IT';
```

✓ Gets IT employees and their salaries.

♦ 4. Sort Results with ORDER BY

sql

```
SELECT first_name, salary
FROM employees
ORDER BY salary DESC;
```

✓ Lists employees from highest to lowest salary.

♦ 5. Aggregate Data with GROUP BY

sql

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

✓ Shows average salary for each department.

Operators in sql

1. Arithmetic operators

+, -, *, /

2. Relational operators

=, != or <>, <, <=, >, >=

3. Logical operators

AND, OR, NOT

4. Special operators

IN, BETWEEN-AND, LIKE, IS NULL

5. Set operators

UNION, UNION ALL, INTERSECT, MINUS

6. Concatenation operators

||

✓ Arithmetic Operators in SQL

SQL provides **arithmetic operators** to perform mathematical operations on numeric values in queries, expressions, and calculations.

♦ List of Arithmetic Operators

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b

1. + (Addition)

Adds two numbers.

sql

```
SELECT salary, salary + 1000 AS increased_salary  
FROM employees;
```

✓ This query adds 1000 to each employee's salary.

♦ 2. - (Subtraction)

Subtracts the second number from the first.

```
sql  
SELECT salary, salary - 500 AS adjusted_salary  
FROM employees;
```

✓ Subtracts 500 from each employee's salary.

♦ 3. * (Multiplication)

Multiplies two numbers.

```
sql  
SELECT salary, salary * 2 AS double_salary  
FROM employees;
```

✓ Multiplies each salary by 2.

♦ 4. / (Division)

Divides the first number by the second.

sql

```
SELECT salary, salary / 2 AS half_salary  
FROM employees;
```

✓ Divides each salary by 2.

♦ Combined Example

You can use multiple arithmetic operators in the same query:

sql

```
SELECT  
    salary,  
    salary + 1000 - 200 AS bonus_adjusted_salary,  
    salary * 1.1 AS salary_with_10_percent_raise,  
    salary / 12 AS monthly_salary  
FROM employees;
```

✓ This query:

- Adds 1000 and subtracts 200 (bonus logic),
- Increases salary by 10%,
- Calculates monthly salary assuming 12 months.

✓ Relational Operators in SQL

In **SQL**, **relational operators** (also called **comparison operators**) are used to compare two values. These operators return a Boolean value: TRUE, FALSE, or NULL (unknown).

♦ List of Relational Operators

Operator	Description	Example
=	Equal to	salary = 5000
!= or <>	Not equal to	salary != 5000
>	Greater than	salary > 5000
<	Less than	salary < 5000
>=	Greater than or equal to	salary >= 5000
<=	Less than or equal to	salary <= 5000

♦ Examples Using the employees Table

1. Equality

```
sql
SELECT * FROM employees
WHERE department_id = 10;
```

✓ Returns all employees in department 10.

2. Not Equal

sql

```
SELECT * FROM employees  
WHERE job_id <> 'IT_PROG';
```

✓ Returns all employees who are **not** IT programmers.

3. Greater Than

sql

```
SELECT * FROM employees  
WHERE salary > 6000;
```

✓ Returns employees with a salary **greater than 6000**.

4. Less Than or Equal

sql

```
SELECT * FROM employees  
WHERE salary <= 4000;
```

✓ Returns employees with a salary **less than or equal to 4000**.

✓ Logical Operators in SQL

In SQL, **logical operators** are used to combine multiple conditions in a WHERE clause or to control the flow of logical expressions. The main logical operators in Oracle SQL are:

♦ 1. AND

- Returns TRUE if **both** conditions are true.

Example:

```
sql
SELECT * FROM employees
WHERE department_id = 10 AND salary > 5000;
```

♦ 2. OR

- Returns TRUE if **either** condition is true.

Example:

```
sql
SELECT * FROM employees
WHERE department_id = 10 OR salary > 5000;
```

♦ 3. NOT

- Negates a condition; returns TRUE if the condition is **false**.

Example:

```
sql
SELECT * FROM employees
WHERE NOT department_id = 10;
```

Special Operators in SQL

1. IN

Checks if a value is in a list of values or a subquery result.

Example:

```
sql
SELECT * FROM employees
WHERE department_id IN (10, 20, 30);
```

Note: Be careful with IN and NULL – if the list contains NULL, the result may be unknown.

2. BETWEEN ... AND

Checks if a value is in a specified inclusive range.

Example:

```
sql
SELECT * FROM employees
WHERE salary BETWEEN 3000 AND 7000;
```

3. LIKE

Used for **pattern matching** with wildcards:

- % matches any sequence of characters.
- _ matches a single character.

Example:

sql

```
SELECT * FROM employees
```

```
WHERE last_name LIKE 'Sm%'; -- Matches 'Smith', 'Smythe', etc.
```

4. IS NULL / IS NOT NULL

Tests whether a value is NULL.

Example:

sql

```
SELECT * FROM employees
```

```
WHERE commission_pct IS NULL;
```

Set Operators in SQL

In SQL, **set operators** are used to **combine the results of two or more SELECT statements** into a single result set. They work similarly to mathematical set operations like **UNION**, **INTERSECT**, and **MINUS**.

◆ List of Set Operators in Oracle SQL

Operator	Description
UNION	Combines results from two queries and removes duplicates
UNION ALL	Combines all results from two queries including duplicates
INTERSECT	Returns only the common rows between two queries
MINUS	Returns rows from the first query that are not in the second

◆ Rules for Using Set Operators

1. Both SELECT statements must have the **same number of columns**.
2. The **data types** of corresponding columns must be **compatible**.
3. The **column order** must match.
4. Set operators must appear **outside of subqueries**, not inside them directly.

♦ 1. UNION – Combine and Remove Duplicates

sql

```
SELECT department_id FROM employees
```

```
UNION
```

```
SELECT department_id FROM departments;
```

✓ Returns unique department IDs from both tables.

♦ 2. UNION ALL – Combine and Keep Duplicates

sql

```
SELECT department_id FROM employees
```

```
UNION ALL
```

```
SELECT department_id FROM departments;
```

✓ Returns all department IDs including duplicates.

♦ 3. INTERSECT – Return Common Rows Only

sql

```
SELECT department_id FROM employees
```

INTERSECT

```
SELECT department_id FROM departments;
```

✓ Returns only the department IDs that exist in **both** tables.

♦ 4. MINUS – Return Rows from First Query Only

sql

```
SELECT department_id FROM departments
```

MINUS

```
SELECT department_id FROM employees;
```

✓ Returns department IDs that exist in departments but **not** in employees.

✓ Concatenation Operators in SQL

In Oracle SQL, the **concatenation operator** is used to **join two or more strings together**.

✓ **Syntax:**

sql

```
string1 || string2 [|| string3 ...]
```

- The || operator joins strings together.
- It automatically **converts numbers or dates to strings** when used.

Example : Basic String Concatenation

sql

```
SELECT 'Hello' || ' ' || 'World' AS greeting FROM dual;
```

Result:

markdown

GREETING

Hello World

◆ Transaction Control Language

In SQL, **TCL (Transaction Control Language)** is used to **manage transactions** in a database. Transactions are logical units of work that contain one or more SQL statements, and TCL commands let you control when these changes are saved (committed) or undone (rolled back).

Main TCL Commands in SQL

Command	Description
COMMIT	Saves all changes made in the transaction to the database permanently
ROLLBACK	Undoes changes made in the current transaction
SAVEPOINT	Sets a point within a transaction to which you can later roll back

1. COMMIT

Permanently saves changes made by INSERT, UPDATE, or DELETE.

Example:

```
sql
```

```
INSERT INTO employees (employee_id, first_name) VALUES (999,  
'Test');
```

```
COMMIT;
```

- Now the new row is saved and visible to others.

2. ROLLBACK

Cancels all changes made in the current transaction (since the last COMMIT or SAVEPOINT).

✓ **Example:**

```
sql
```

```
DELETE FROM employees WHERE employee_id = 999;
```

```
ROLLBACK;
```

- The deletion is canceled.
-

3. SAVEPOINT

Creates a named point in a transaction you can roll back to **without undoing the entire transaction**.

✓ **Example:**

```
sql
```

```
SAVEPOINT sp1;
```

```
UPDATE employees SET salary = salary + 1000 WHERE department_id = 10;
```

```
SAVEPOINT sp2;
```

```
UPDATE employees SET salary = salary + 500 WHERE department_id = 20;
```

```
ROLLBACK TO sp2; -- Only undoes the last update
```

✓ Combined TCL Example

Let's say Alice transfers 200 to Bob, and we want full control over this transaction with SAVEPOINT, ROLLBACK, and COMMIT.

```
sql

-- Start transaction (Oracle begins automatically after last
COMMIT)

UPDATE accounts SET balance = balance - 200 WHERE account_id = 1;

-- Save point after deducting from Alice

SAVEPOINT after_alice_deduction;

UPDATE accounts SET balance = balance + 200 WHERE account_id = 2;

-- Oops! Let's say there's a mistake (e.g., wrong account or
wrong amount)

-- We decide to roll back only to the point after Alice's
deduction

ROLLBACK TO after_alice_deduction;

-- Now fix the mistake and re-add the correct amount to Bob

UPDATE accounts SET balance = balance + 200 WHERE account_id = 2;
```

```
-- Commit the whole transaction
```

```
COMMIT;
```

Step	Action
1	Alice's balance is reduced by 200
2	Savepoint created (after_alice_deduction)
3	Amount added to Bob, but we realize an issue
4	Rollback to savepoint: undo only Bob's credit
5	Re-credit Bob correctly
6	Commit the entire transaction

♦ **Final Account Balances:**

- Alice: 800
 - Bob: 1700
-

Note:

If you had used ROLLBACK without a savepoint, **both** updates (to Alice and Bob) would have been undone.

◆ **Data Control Language**

In SQL, **DCL (Data Control Language)** is used to **control access to data** in the database. DCL commands manage **privileges** and **permissions** for users and roles.

Main DCL Commands in SQL

Command	Description
GRANT	Gives privileges to users or roles
REVOKE	Removes privileges from users or roles

◆ **1. GRANT – Give Privileges**

The GRANT command is used to **assign access rights** to database objects (like tables, views, procedures) or system privileges.

✓ **Syntax:**

sql

```
GRANT privilege_type ON object TO user_or_role;
```

♦ **Example 1: Granting access to a table**

sql

```
GRANT SELECT, INSERT ON employees TO hr_user;
```

✓ This allows hr_user to select and insert into the employees table.

♦ **Example 2: Granting system privileges**

sql

```
GRANT CREATE SESSION TO hr_user;
```

✓ Allows the user to log in to the database.

♦ **Example 3: Grant with WITH GRANT OPTION**

sql

```
GRANT SELECT ON employees TO hr_user WITH GRANT OPTION;
```

✓ Allows hr_user to give that same SELECT privilege to other users.

♦ 2. REVOKE – Remove Privileges

The REVOKE command is used to **take back privileges** granted earlier.

✓ Syntax:

sql

```
REVOKE privilege_type ON object FROM user_or_role;
```

♦ Example 1: Revoking object privileges

sql

```
REVOKE SELECT, INSERT ON employees FROM hr_user;
```

✓ Removes SELECT and INSERT rights from hr_user.

♦ Example 2: Revoking system privileges

sql

```
REVOKE CREATE SESSION FROM hr_user;
```

✓ Prevents hr_user from logging into the database.

♦ Object vs System Privileges

Type	Example Privileges
System	CREATE SESSION, CREATE TABLE, DROP USER
Object	SELECT, INSERT, UPDATE, DELETE on specific tables or views

⚠ Notes:

- Only a **DBA** or a user with **GRANT ANY PRIVILEGE** can grant system privileges.
- **Revoking** a privilege with **WITH GRANT OPTION** can **cascade** and remove access from users who received it indirectly.
- DCL commands like **GRANT** and **REVOKE** cause an **implicit commit**.

◆ GROUP BY

In **SQL**, the **GROUP BY** clause is used to **aggregate data** based on one or more columns. It groups rows that have the same values in specified columns into **summary rows**, typically in combination with **aggregate functions** like **SUM()**, **COUNT()**, **AVG()**, **MAX()**, or **MIN()**.

◆ Syntax

sql

```
SELECT column1, column2, AGGREGATE_FUNCTION(column3)
FROM table_name
GROUP BY column1, column2;
```

- column1, column2 → grouping columns
- AGGREGATE_FUNCTION(column3) → an aggregate function applied to other column(s)

🎓 Suppose you have this table called marks:

student_id	subject	marks
1	Math	85
1	English	78
2	Math	92
2	English	81
3	Math	75
3	English	88

Example 1: Total Marks per Student

```
sql
```

```
SELECT student_id, SUM(marks) AS total_marks
```

```
FROM marks
```

```
GROUP BY student_id;
```

Output:

student_id	total_marks
1	163
2	173
3	163

Example 2: Average Marks per Subject

```
sql
```

```
SELECT subject, AVG(marks) AS avg_marks  
  
FROM marks  
  
GROUP BY subject;
```

Output:

subject	avg_marks
Math	84.0

English	82.3
---------	------

Example 3: Count of Students per Subject

sql

```
SELECT subject, COUNT(student_id) AS student_count
```

```
FROM marks
```

```
GROUP BY subject;
```

Output:

subject	student_count
Math	3
English	3

Example 4: Using HAVING – Subjects with Average Marks > 83

sql

```
SELECT subject, AVG(marks) AS avg_marks
```

```
FROM marks
GROUP BY subject
HAVING AVG(marks) > 83;
```

Output:

subject	avg_marks
Math	84.0

 Table: marks

student_id	subject	marks	semester
1	Math	85	1
1	English	78	1
2	Math	92	1
2	English	81	1
3	Math	75	2
3	English	88	2
4	Math	60	2
4	English	72	2

 Example 1 : Average Marks per Subject for Semester 1, Ordered by Average Descending

♦ **SQL Query:**

sql

```
SELECT subject, AVG(marks) AS avg_marks  
FROM marks  
WHERE semester = 1  
GROUP BY subject  
ORDER BY avg_marks DESC;
```

♦ Step-by-Step Explanation:

Clause	Purpose
SELECT	What columns to display (subject, AVG(marks))
FROM	From which table (marks)
WHERE	Filter only records from semester = 1
GROUP BY	Group rows by subject to calculate average marks
ORDER BY	Sort the results by avg_marks in descending order

♦ Output:

subject	avg_marks
Math	88.5
English	79.5

✅ **Example 2: Total Marks per Student in Semester 2, Ordered by Student ID**

sql

```
SELECT student_id, SUM(marks) AS total_marks
FROM marks
WHERE semester = 2
GROUP BY student_id
ORDER BY student_id;
```

◆ Output:

student_id	total_marks
3	163
4	132

◆ **HAVING CLAUSE**

The HAVING clause in SQL is used to filter groups of rows after applying the GROUP BY clause. It's like a WHERE clause, but specifically for aggregate results (like SUM(), AVG(), COUNT(), etc.).

◆ Syntax

sql

```
SELECT column1, AGGREGATE_FUNCTION(column2)
FROM table_name
WHERE condition
GROUP BY column1
HAVING AGGREGATE_FUNCTION(column2) condition;
```

◆ Example Using marks Table

Suppose you have this table:

student_id	subject	marks	semester
1	Math	85	1
1	English	78	1
2	Math	92	1
2	English	81	1
3	Math	75	2
3	English	88	2
4	Math	60	2
4	English	72	2

✓ Example 1: Get Subjects with Average Marks > 80

```
sql
SELECT subject, AVG(marks) AS avg_marks
FROM marks
GROUP BY subject
HAVING AVG(marks) > 80;
```

Result:

subject	avg_marks
Math	78.0
English	79.75

(Assuming actual values—adjust depending on your data.)

✓ Example 2: Total Marks per Student in Semester 2, Only If Total > 150

```
sql
SELECT student_id, SUM(marks) AS total_marks
FROM marks
WHERE semester = 2
GROUP BY student_id
HAVING SUM(marks) > 150;
```

Result:

student_id	total_marks
3	163

Key Difference Between WHERE and HAVING:

Clause	Filters...	Can use aggregates?
WHERE	Before grouping	✗ No
HAVING	After grouping	✓ Yes

◆ OFFSET AND FETCH

In **SQL**, the **OFFSET** and **FETCH** clauses are used for **pagination**—retrieving a subset of rows from a result set, usually for things like pages of results in an app or website.

OFFSET

In **Oracle SQL**, the **OFFSET** clause is used to **skip a specific number of rows** in the result set. It is typically used in combination with **ORDER BY** and optionally **FETCH** to implement **pagination**.

Syntax

sql

SELECT columns

FROM table_name

ORDER BY column

OFFSET n ROWS;

- **OFFSET n ROWS** skips the **first n rows**
- You must use **ORDER BY** to ensure a deterministic result

♦ **Example: Skip First 3 Rows from a students Table**

Assume the students table:

student_id	name	marks
1	Alice	90
2	Bob	85
3	Carol	88
4	David	92
5	Emma	84

♦ **Query: Skip First 3 Rows by Marks (Descending)**

sql

```
SELECT student_id, name, marks
```

```
FROM students
```

```
ORDER BY marks DESC
```

```
OFFSET 3 ROWS;
```

Result:

student_id	name	marks
------------	------	-------

2	Bob	85
---	-----	----

5	Emma	84
---	------	----

FETCH

The FETCH clause in **Oracle SQL** is used to **limit the number of rows returned** by a query. It is commonly paired with OFFSET to implement **pagination**.

◆ Syntax

sql

SELECT columns

FROM table_name

ORDER BY column

FETCH FIRST n ROWS ONLY;

or with pagination:

sql

SELECT columns

```
FROM table_name
```

```
ORDER BY column
```

```
OFFSET x ROWS
```

```
FETCH NEXT y ROWS ONLY;
```

◆ Example: Top 3 Students by Marks

Assume a table students

student_id	name	marks
1	Alice	90
2	Bob	85
3	Carol	88
4	David	92
5	Emma	84

✓ Query: Get Top 3 Students

sql

```
SELECT student_id, name, marks
```

```
FROM students
```

```
ORDER BY marks DESC
```

```
FETCH FIRST 3 ROWS ONLY;
```

Result:

student_id	name	marks
4	David	92
1	Alice	90
3	Carol	88

◆ FETCH with OFFSET (Pagination)

sql

```
-- Get students ranked 4-5 by marks
```

```
SELECT student_id, name, marks
```

```
FROM students
```

ORDER BY marks DESC

OFFSET 3 ROWS

FETCH NEXT 2 ROWS ONLY;

◆ FETCH Variants

Clause	Description
FETCH FIRST n ROWS ONLY	Returns only the first n rows
FETCH NEXT n ROWS ONLY	Same as above, used after OFFSET
FETCH FIRST n ROWS WITH TIES	Includes all rows that tie with the last row

Join operations in SQL

In Oracle SQL, JOIN operations are used to combine rows from two or more tables based on a related column. This is essential for working with relational data.

Here's clear example of a JOIN operation in Oracle SQL using two tables.

Tables

employees table:

emp_id	emp_name
--------	----------

1	Alice
---	-------

2	Bob
---	-----

3	Carol
---	-------

departments table:

dept_id	emp_id	dept_name
---------	--------	-----------

10	1	HR
----	---	----

20	2	Finance
----	---	---------

JOIN Example: Show Employee Names and Their Departments

sql

```
SELECT e.emp_name, d.dept_name  
  
FROM employees e  
  
JOIN departments d  
  
ON e.emp_id = d.emp_id;
```

Result:

emp_name	dept_name
----------	-----------

Alice	HR
-------	----

Bob	Finance
-----	---------

♦ Explanation:

- This is an **INNER JOIN**: it returns only employees who have a matching department.
 - Carol is not included because she doesn't belong to any department in the departments table.
-



Types of JOINS in Oracle SQL

JOIN Type	Description
INNER JOIN	Returns rows that have matching values in both tables
OUTERJOIN	An outer join returns all matching rows plus non-matching rows from one or both tables , with NULL for missing values.
CROSS JOIN	Returns the Cartesian product of the two tables
SELF JOIN	Joins a table to itself

INNER JOIN:

An inner join (sometimes called a simple join) is a join of two or more tables that returns only those rows that satisfy the join condition.

1. Equijoin

2. Nonequijoin

1. EQUIJOIN in Oracle SQL

An **Equijoin** is a type of JOIN that combines rows from two or more tables **based on an equality condition** between specified columns—usually using the = operator.

In Oracle SQL, an **Equijoin** is typically written using either:

- **Oracle legacy syntax:** using the WHERE clause
- **ANSI syntax** (recommended): INNER JOIN ... ON

Oracle legacy syntax:

Example Tables

Table: students

student_id	name
1	Alice
2	Bob
3	Carol

Table: marks

student_id	subject	marks
------------	---------	-------

1	Math	90
---	------	----

2	Science	85
---	---------	----

♦ Oracle-Style Equijoin Query:

sql

```
SELECT s.student_id, s.name, m.subject, m.marks
```

```
FROM students s, marks m
```

```
WHERE s.student_id = m.student_id;
```

✓ Output:

student_id	name	subject	marks
------------	------	---------	-------

1	Alice	Math	90
---	-------	------	----

2	Bob	Science	85
---	-----	---------	----

Explanation:

- The WHERE `s.student_id = m.student_id` is the **equijoin condition**.
- It returns only rows where `student_id` exists in **both tables**.
- This is functionally the same as using `INNER JOIN ... ON`, but written in **Oracle's legacy syntax**.

Note: It changes the orders of records in result each time. It follows the join column order sometimes and first column order sometimes so on...if you want specific order then use order by.

ANSI syntax:

◆ Example Scenario

Let's say we have two tables:

employees table:

emp_id	emp_name
--------	----------

1	Alice
---	-------

2	Bob
---	-----

3 Carol

departments table:

dept_id	emp_id	dept_name
---------	--------	-----------

10	1	HR
----	---	----

20	2	Finance
----	---	---------

✓ Example 1: Equijoin Using ANSI JOIN Syntax

sql

```
SELECT e.emp_name, d.dept_name
```

```
FROM employees e
```

```
JOIN departments d
```

```
ON e.emp_id = d.emp_id;
```

Result:

emp_name	dept_name
----------	-----------

Alice HR

Bob Finance

2. Nonequijoin:

A Nonequijoin is a join with a join condition containing all operators other than equality operator.

Nonequi Conditions

- >
- <
- >=
- <=
- != or <> (though rarely useful in joins)

1 Nonequi Join Using >

Scenario: Employees matched to salary tiers where employee salary is greater than the tier minimum.

sql

```
SELECT e.emp_name, e.salary, t.tier_name
```

```
FROM employees e, salary_tiers t
```

```
WHERE e.salary > t.min_salary;
```

- **Explanation:** Join employees with all tiers having a min_salary less than employee's salary.
-

② Nonequi Join Using <

Scenario: Products matched to price categories where product price is less than the category max price.

sql

```
SELECT p.product_name, p.price, c.category_name
```

```
FROM products p, price_categories c
```

```
WHERE p.price < c.max_price;
```

- **Explanation:** Join products with categories where price is below max price.
-

③ Nonequi Join Using <> (Not Equal)

Scenario: Find all pairs of students where student IDs are not equal (i.e., different students).

sql

```
SELECT s1.student_name AS student1, s2.student_name AS student2
FROM students s1, students s2
WHERE s1.student_id <> s2.student_id;
```

- **Explanation:** Self-join excluding pairs with the same student.
-

Summary

Operator	Usage	Example Condition
>	Greater than	e.salary > t.min_salary
<	Less than	p.price < c.max_price
<>	Not equal (different values)	s1.student_id <> s2.student_id

Band join:

A Band join is a special type of nonequi join in which key values in one data set must fall within the specified range ("band") of the second data set.

How Does a Band Join Work?

Suppose you have:

- A table A with a value column (e.g., salary)
- A table B with two columns defining a range: min_value and max_value

You want to join rows from A to B where:

`css`

`A.value BETWEEN B.min_value AND B.max_value`

This is the band join condition because the join matches rows where A.value lies within the band (range) defined by B.

Why Use Band Joins?

- Useful for mapping values to ranges (e.g., employee salaries to salary grades, transaction amounts to discount tiers).
 - It's a natural way to do range-based categorization or bucketing.
 - More flexible than simple equi-joins.
-

Oracle Style Band Join Example (Summary)

sql

```
SELECT a.column1, b.column2
```

```
FROM table_a a, table_b b
```

```
WHERE a.value_column BETWEEN b.min_value AND b.max_value;
```

Important Notes

- Because the join condition is not equality, the result can have multiple matches per row from the first table (if ranges overlap).
- Oracle style uses the WHERE clause, not the ANSI JOIN syntax.
- You can also express the band join using two conditions:

sql

WHERE a.value_column >= b.min_value

AND a.value_column <= b.max_value

OUTER JOIN :

An outer join returns all matching rows plus non-matching rows from one or both tables, with NULL for missing values.

1. LEFT OUTERJOIN:
2. RIGHT OUTERJOIN
3. FULL OUTERJOIN

♦ 1. Oracle Style (Proprietary Syntax using (+))

➤ Left Outer Join:

Returns all rows from the left table, even if there's no match in the right table.

Syntax:

sql

SELECT e.emp_name, d.dept_name

FROM employees e, departments d

WHERE e.dept_id = d.dept_id(+);

● Meaning: Show all employees, even if they have no department.

➤ Right Outer Join:

Returns all rows from the right table, even if there's no match in the left table.

Syntax:

sql

```
SELECT e.emp_name, d.dept_name
```

```
FROM employees e, departments d
```

```
WHERE e.dept_id(+) = d.dept_id;
```

● Meaning: Show all departments, even if no employee is assigned.

❌ Oracle's (+) syntax does not support full outer joins.

➤ Full Outer Join: Using UNION

syntax

sql

```
-- LEFT OUTER JOIN part (all employees, with departments if  
matched)
```

```
SELECT e.emp_name, d.dept_name
```

```
FROM employees e, departments d
```

```
WHERE e.dept_id = d.dept_id(+)
```

```
UNION
```

```
-- RIGHT OUTER JOIN part (all departments, with employees if  
matched)
```

```
SELECT e.emp_name, d.dept_name
```

```
FROM employees e, departments d
```

```
WHERE e.dept_id(+) = d.dept_id;
```

Explanation

- The first query gets all employees (even if they have no department).
- The second query gets all departments (even if they have no employees).
- UNION removes duplicates and combines the result.

✅ This effectively mimics a FULL OUTER JOIN.

♦ 2. Ansi style

➤ Left Outer Join:

Returns all rows from the left table and matching rows from the right table. If no match, fills with NULL.

syntax:

sql

```
SELECT e.emp_name, d.dept_name
```

```
FROM employees e
```

```
LEFT OUTER JOIN departments d
```

```
ON e.dept_id = d.dept_id;
```

● Keeps all employees, even if they don't belong to any department.

➤ Right Outer Join:

Returns all rows from the right table and matching rows from the left table.

syntax:

sql

```
SELECT e.emp_name, d.dept_name  
  
FROM employees e  
  
RIGHT OUTER JOIN departments d  
  
    ON e.dept_id = d.dept_id;
```

- Keeps all departments, even those with no employees.
-

➤ Full Outer Join:

Returns all rows from both tables. Where there's no match, NULL fills the missing side.

syntax:

```
sql  
  
SELECT e.emp_name, d.dept_name  
  
FROM employees e  
  
FULL OUTER JOIN departments d  
  
    ON e.dept_id = d.dept_id;
```

- Keeps all employees and all departments, matched or not.
-

✓ ANSI Style Benefits

- Easier to read and maintain
- Supports all outer join types (Oracle's (+) style does not support FULL OUTER JOIN)
- Works across all modern SQL platforms

Self Join:

A self join is a join of a table to itself. This table appears twice in the FROM clause and is followed by table aliases that qualify column names in the join condition.

✓ When to Use a Self Join

Use a self join when:

- A table has a **hierarchical** or **referential** relationship within itself.
- You need to compare **rows in the same table**, such as:
 - Employees and their managers
 - Products and similar products
 - Students and their mentors

✓ What Does != Do in a Self Join?

- It returns pairs of **different rows** from the same table.
- Common use cases include:
 - Finding **all possible combinations** of distinct rows.
 - Comparing rows **excluding self** (e.g., pairing students who aren't the same person).

♦ Oracle SQL (Proprietary Style)

Here's a simple self join using `!=` in **Oracle-style** SQL:

sql

```
SELECT s1.student_name AS student1,  
       s2.student_name AS student2  
FROM students s1, students s2  
WHERE s1.student_id != s2.student_id;
```



Example Table: students

student_id	student_name
1	Alice

2 Bob

3 Charlie

✓ Output (All Pairs of Different Students)

STUDENT1	STUDENT2
----------	----------

Alice	Bob
-------	-----

Alice	Charlie
-------	---------

Bob	Alice
-----	-------

Bob	Charlie
-----	---------

Charlie	Alice
---------	-------

Charlie	Bob
---------	-----

Explanation

- The self join creates all possible **row combinations**.
 - The condition `s1.student_id != s2.student_id` ensures that a student is **not paired with themselves**.
-

♦ **Ansi style**

♦ **Example:** Compare Students Who Are Not the Same

🎓 **Table: students**

sql

```
CREATE TABLE students (  
  
    student_id    NUMBER,  
  
    student_name  VARCHAR2(50)  
  
);
```

📌 **Sample Data**

student_id	student_name
1	Alice

2 Bob

3 Charlie

- ♦ ANSI Style Self Join Using !=

sql

```
SELECT s1.student_name AS student1,  
       s2.student_name AS student2  
  
FROM students s1  
  
JOIN students s2  
  
ON s1.student_id != s2.student_id;
```



Output

STUDENT1 STUDENT2

Alice Bob

Alice Charlie

Bob Alice

Bob Charlie

Charlie Alice

Charlie Bob

Explanation

- The table students is joined to itself using aliases s1 and s2.
- ON s1.student_id != s2.student_id ensures you exclude self-pairs.
- This gives all unique pairings of different students.

Cross join/cartesian join :

If two tables in a join query have no join condition, then oracle database returns their cartesian product.

✓ What It Does

- No join condition is used.
- Total number of rows in the result = rows in table A × rows in table B.



Example Scenario

Let's say we want to generate **all combinations of colors and sizes** for a product catalog.

◆ Step 1: Create Sample Tables

sql

```
CREATE TABLE colors (  
  
    color VARCHAR2(20)  
  
);
```

```
CREATE TABLE sizes (  
  
    size VARCHAR2(20)  
  
);
```

```
-- Insert some sample data
```

```
INSERT INTO colors VALUES ('Red');
```

```
INSERT INTO colors VALUES ('Blue');
```

```
INSERT INTO sizes VALUES ('Small');
```

```
INSERT INTO sizes VALUES ('Medium');
```

```
INSERT INTO sizes VALUES ('Large');
```

♦ Step 2: ANSI Style CROSS JOIN

```
sql
```

```
SELECT c.color, s.size
```

```
FROM colors c
```

```
CROSS JOIN sizes s;
```

✓ This returns all combinations of color × size.

♦ Output

COLOR	SIZE
-------	------

Red	Small
-----	-------

Red Medium

Red Large

Blue Small

Blue Medium

Blue Large

♦ Step 3: Oracle Proprietary Style CROSS JOIN

sql

```
SELECT c.color, s.size
```

```
FROM colors c, sizes s;
```

✅ This is the **Oracle-style** (non-ANSI) cross join – same result, but written with comma-separated tables and **no WHERE clause**.

⚠️ If you forget a WHERE clause when doing normal joins using Oracle-style, you might **accidentally perform a cross join**.

✅ Summary

Style	Syntax	Supported in Oracle?
ANSI	FROM A CROSS JOIN B	✓ Yes
Oracle (legacy)	FROM A, B (no WHERE)	✓ Yes



Introduction to Subqueries in SQL

A **subquery** is a query **nested inside** another SQL query – typically in the SELECT, FROM, or WHERE clause.

It helps you **retrieve intermediate results** used by the outer (main) query.

✓ What Is a Subquery?

A **subquery** is a SQL query nested **inside** another query's SELECT, FROM, or WHERE clause.

It helps break down complex logic into manageable parts.

◆ Basic Structure

sql

```
SELECT column1
```

FROM table

WHERE column2 = (SELECT column3 FROM another_table WHERE condition);

Why Use Subqueries?

- To **filter results** based on values from another table
 - To **calculate derived values** (e.g. maximum, average)
 - To **create virtual tables** to join with
 - To **improve readability** of complex logic
-



Example 1: Subquery in WHERE(nested subquery)

Get employees with the **highest salary**:

sql

SELECT emp_name, salary

FROM employees

WHERE salary = (SELECT MAX(salary) FROM employees);

- In the Where clause we can write only 255 levels of subqueries.
-



Example 2: Subquery in FROM (Inline View)

Get average salary per department:

sql

```
SELECT dept_id, AVG(salary)
FROM (SELECT * FROM employees WHERE active = 'Y')
GROUP BY dept_id;
```

- In the From clause we can write unlimited subqueries.
-



Example 3: Subquery in SELECT

sql

```
SELECT emp_name,
       (SELECT dept_name FROM departments d WHERE d.dept_id = e.dept_id) AS
dept
FROM employees e;
```



Summary

Feature	Description
Purpose	Use result of one query inside another
Use cases	Filtering, calculating, joining
Benefits	More powerful and readable SQL



When Should We Use a Subquery?

Use a subquery in Oracle SQL when:

Situation	Why a Subquery Helps
You need a value that depends on another table	Subqueries return scalar or tabular values dynamically
You want to filter based on an aggregated value	Use subquery in WHERE to compare with MAX(), etc.
You want to compare across tables	Use subquery with IN, EXISTS, =, ANY, ALL

You want to simplify complex joins

Use subqueries in FROM to isolate logic

You need conditional logic per row

Use **correlated subqueries**

Use subqueries for the following purposes:

- To define the set of rows to be inserted into the target table of an insert or create table statement.

INSERT INTO ... SELECT – Use a Subquery to Insert Rows

Syntax:

sql

```
INSERT INTO target_table (column1, column2, ...)
```

```
SELECT column1, column2, ...
```

```
FROM source_table
```

```
WHERE condition;
```

- To define the set of rows to be inserted in a view or materialized view in a create view or create materialized view statement

1. CREATE VIEW with a Subquery

Syntax:

sql

```
CREATE VIEW view_name AS

SELECT column1, column2, ...

FROM table1

WHERE condition;
```

Example: Create a view of high-salary employees

```
sql

CREATE VIEW high_salary_ems AS

SELECT emp_id, emp_name, salary

FROM employees

WHERE salary > 5000;
```

 The **subquery (SELECT ...)** defines the rows included in the view.

- To define one or more values to be assigned to existing rows in an Update statement

Update Statement with Subquery

```
sql

UPDATE employees e

SET dept_name = (

    SELECT d.dept_name
```

```
FROM departments d

WHERE d.dept_id = e.dept_id

);
```

✓ This subquery pulls the correct department name for each employee based on dept_id.

- To provide values for conditions in a where clause, having clause, or start with clause of select, update and delete statements

1. Subquery in WHERE Clause

Example: Get employees who work in departments located in 'New York'

```
sql

SELECT emp_name

FROM employees

WHERE dept_id IN (

    SELECT dept_id

    FROM departments

    WHERE location = 'New York'

);
```

📌 The subquery returns department IDs to match in the outer query.

2. Subquery in HAVING Clause

Example: Get departments where the average salary is greater than the company-wide average

```
sql

SELECT dept_id, AVG(salary) AS avg_salary

FROM employees

GROUP BY dept_id

HAVING AVG(salary) > (

    SELECT AVG(salary) FROM employees

);
```

📌 The subquery calculates the company-wide average salary.

3. Subquery in START WITH Clause (Hierarchical Queries)

Example: Get all employees under the same manager as the one with the highest salary

```
sql

SELECT emp_id, emp_name, manager_id

FROM employees
```

```
START WITH emp_id = (  
  
    SELECT emp_id  
  
    FROM employees  
  
    WHERE salary = (SELECT MAX(salary) FROM employees)  
  
)  
  
CONNECT BY PRIOR emp_id = manager_id;
```

📌 The subquery in START WITH finds the employee who has the highest salary – then Oracle follows the hierarchy down from that employee.

4. Subquery in DELETE or UPDATE with WHERE

Example: Delete employees in departments that are marked inactive

```
sql  
  
DELETE FROM employees  
  
WHERE dept_id IN (  
  
    SELECT dept_id  
  
    FROM departments  
  
    WHERE active = 'N'  
  
);
```

📌 Subquery returns only the inactive department IDs.

- To define a table to be operated on by a containing query

Types of subqueries :

A subquery is a query nested inside another SQL query – used to return data that helps the main (outer) query make decisions. Subqueries are powerful tools that help you write modular, readable, and efficient SQL.

1. Non-correlated :

A **non-correlated subquery** is a subquery that **does not reference any column from the outer query**.

It is **self-contained** and **executes only once**, and its result is then used by the main (outer) query.

♦ Key Characteristics

Feature	Description
Independent	Does not depend on the outer query
Executes Once	Runs only once , not per row
Flexible Use	Can be used in SELECT, FROM, WHERE, HAVING

Efficient

Often more efficient than correlated subqueries

2. Correlated :

A **correlated subquery** is a subquery that **references columns from the outer query**.

It is **executed once for every row** processed by the outer query – making it dependent on the outer query's current row.

♦ Key Characteristics

Feature	Description
Depends on outer query	Uses a value from the outer query inside itself
Executes repeatedly	Runs once for each row in the outer query
Often used for row-by-row comparisons	Ideal for comparisons within groups

Non-correlated subqueries:

In Oracle SQL, a **non-correlated subquery** is a subquery that is **independent** of the outer query – it can be executed on its own without referencing any columns from the outer query.

- ♦ **Characteristics of a Non-Correlated Subquery:**

- Executed **once** for the entire outer query.
 - Does **not** reference columns from the outer query.
 - Can be placed in the SELECT, FROM, or WHERE clause.
-

♦ Syntax Example

sql

```
SELECT first_name, last_name
```

```
FROM employees
```

```
WHERE department_id = (
```

```
    SELECT department_id
```

```
    FROM departments
```

```
    WHERE department_name = 'IT'
```

```
);
```

Explanation:

The subquery:

sql

```
SELECT department_id FROM departments WHERE department_name = 'IT'
```

- is **non-correlated** because it doesn't use any columns from the outer employees query.

- It returns the department ID for 'IT' once, and that result is used to filter the outer query.

Types of subqueries:

According to returned result:

1. Single row subquery
2. Multiple row subquery
3. Multiple column subquery

1.Single row subquery:

In **Oracle SQL**, a **single-row subquery** is a type of subquery that returns **only one row** and **one or more columns**. It's often used in places where only a single value is expected, such as in a **WHERE**, **SELECT**, or **SET** clause.

◆ Key Features of Single-Row Subqueries:

- Returns **only one row**.
 - Can return one or more columns (but commonly one column).
 - Typically used with **comparison operators** like **=**, **<**, **>**, **<=**, **>=**, or **<>**.
-

◆ Example

sql

```
SELECT first_name, last_name, salary
```

```
FROM employees
```

```
WHERE salary > (
```

```
    SELECT salary
```

```
    FROM employees
```

```
    WHERE employee_id = 100
```

```
);
```

Explanation:

The subquery:

sql

```
SELECT salary FROM employees WHERE employee_id = 100
```

- returns **one value (a single salary)**.
- The outer query finds employees whose salary is higher than that salary.

♦ Common Operators Used:

Operator	Meaning
=	Equal to single subquery result

<> Not equal to subquery
result

>, < Greater/Less than subquery
result

>=, <= Greater/Less than or equal

♦ Important Notes:

If the subquery returns **more than one row**, Oracle will throw an error:

sql

ORA-01427: single-row subquery returns more than one row

- To avoid this, ensure the subquery is limited to one row (e.g., using WHERE, ROWNUM, or MAX/MIN).
-

♦ Safe Example Using MAX:

sql

SELECT first_name, last_name, salary

FROM employees

WHERE salary = (

 SELECT MAX(salary)

```
FROM employees  
  
);
```

- This gets the employee(s) with the **highest salary**.
- MAX(salary) returns a **single value**, ensuring no error.

2. Multiple row subquery:

In Oracle SQL, a multiple-row subquery is a subquery that returns more than one row. It's used when the outer query needs to compare a value against a set of values.

◆ Key Features of Multiple-Row Subqueries:

- Returns multiple rows.
 - Must be used with set operators or multi-row comparison operators like:
 - IN
 - ANY / SOME
 - ALL
-

♦ Example 1: Using IN

sql

```
SELECT first_name, last_name
```

```
FROM employees
```

```
WHERE department_id IN (
```

```
    SELECT department_id
```

```
    FROM departments
```

```
    WHERE location_id = 1700
```

```
);
```

Explanation:

- The subquery returns a list of department IDs in location 1700.
- The outer query finds employees who belong to any of those departments.

♦ Example 2: Using ANY

sql

```
SELECT first_name, salary
```

```
FROM employees
```

```
WHERE salary > ANY (
```

```
SELECT salary

FROM employees

WHERE department_id = 50

);
```

Explanation:

- Compares each employee's salary to any salary from department 50.
- Returns employees who earn more than at least one person in department 50.

 ANY and SOME are synonyms in Oracle.

♦ Example 3: Using ALL

```
sql

SELECT first_name, salary

FROM employees

WHERE salary > ALL (

    SELECT salary

    FROM employees

    WHERE department_id = 50

);
```

Explanation:

- Finds employees whose salary is greater than all salaries in department 50.
-

! Caution

Using a comparison operator like = with a multi-row subquery will result in an error:

sql

```
SELECT first_name
```

```
FROM employees
```

```
WHERE department_id = (
```

```
    SELECT department_id
```

```
    FROM departments
```

```
    WHERE location_id = 1700
```

```
);
```

If the subquery returns more than one row, Oracle will raise:

sql

ORA-01427: single-row subquery returns more than one row

✓ To fix it, use IN instead of =.

3. multiple column subquery:

A multiple-column subquery in Oracle SQL returns more than one column per row. You typically use it when the outer query needs to compare against a combination of columns from the subquery.

◆ Key Features of Multiple-Column Subqueries:

- Returns two or more columns per row.
- Can return one or multiple rows.
- Usually used in:
 - WHERE ... IN with row value comparisons
 - EXISTS
 - FROM clause (inline views)

◆ Example 1: Using IN with Multiple Columns

sql

```
SELECT first_name, last_name, department_id, job_id
```

```
FROM employees
```

```
WHERE (department_id, job_id) IN (
```

```
    SELECT department_id, job_id
```

```
FROM job_history  
  
);
```

✓ Explanation:

- The subquery returns pairs of (department_id, job_id) from job_history.
- The outer query returns employees who **have the same department_id and job_id** as in the job history.

♦ Example 2: Using EXISTS with a Multi-Column Condition

sql

```
SELECT first_name, last_name  
  
FROM employees e  
  
WHERE EXISTS (  
  
    SELECT 1  
  
    FROM departments d  
  
    WHERE e.department_id = d.department_id  
  
        AND d.location_id = 1700  
  
);
```

✓ Explanation:

- Even though the subquery isn't returning multiple columns, it uses **multiple columns** in its condition.
 - Returns employees whose department is in location 1700.
-

♦ Example 3: Using Multiple-Column Subquery in FROM (Inline View)

sql

```
SELECT dept_name, employee_count
```

```
FROM (
```

```
    SELECT d.department_name AS dept_name, COUNT(e.employee_id) AS  
employee_count
```

```
    FROM departments d
```

```
    LEFT JOIN employees e ON d.department_id = e.department_id
```

```
    GROUP BY d.department_name
```

```
);
```

✓ Explanation:

- The subquery in the FROM clause returns **two columns**: department name and count of employees.
- The outer query selects from that result.

! Important Notes:

- When using (col1, col2) IN (SELECT col1, col2 ...), the number and order of columns **must match**.
- Oracle compares the combination of values **row by row**.

Correlated subqueries:

A **correlated subquery** in **Oracle SQL** is a subquery that **depends on a column from the outer query**. Unlike a non-correlated subquery, it **cannot run independently**, because it refers to values from the outer query.

◆ Key Characteristics of Correlated Subqueries:

- Executed **once for each row** of the outer query.
- References **one or more columns** from the outer query.
- Often used in WHERE, SELECT, or HAVING clauses.

◆ **Example: Find employees who earn more than the average salary in their department**

sql

```
SELECT first_name, last_name, department_id, salary
```

```
FROM employees e
```

```
WHERE salary > (  
  
    SELECT AVG(salary)  
  
    FROM employees  
  
    WHERE department_id = e.department_id  
  
);
```

✓ Explanation:

The subquery:

```
sql  
SELECT AVG(salary)  
  
FROM employees  
  
WHERE department_id = e.department_id
```

- references e.department_id, which comes from the **outer query**.
- So, the subquery is **re-evaluated for each employee** using their department.

Inline view subquery:

An **inline view** is a **subquery in the FROM clause** that behaves like a temporary table or view for the outer query to use.

✓ **Purpose:** Often used to simplify complex queries, especially when you want to aggregate or filter data before applying additional logic.

◆ Key Features of Inline Views:

- Appears **in the FROM clause**.
 - Returns a **result set** used like a table or view.
 - Can be **aliased** and joined or filtered in the outer query.
 - Helps to **improve readability and structure**.
-

◆ Basic Syntax:

sql

```
SELECT column1, column2
```

```
FROM (
```

```
    SELECT col1, col2
```

```
    FROM table_name
```

```
    WHERE condition
```

```
) inline_view_alias
```

```
WHERE some_condition;
```

◆ Example Scenario:

You have an EMPLOYEES table like this:

EMPLOYEE_ID	FIRST_NAME	SALARY	DEPARTMENT_ID
101	John	8000	10
102	Alice	9000	10
103	Bob	10000	20
104	Carol	7000	10

You want to find all employees who earn **more than the average salary** in their department.

♦ SQL Using Inline View:

sql

```
SELECT e.employee_id,  
       e.first_name,  
       e.salary,  
       d.avg_salary  
FROM employees e  
JOIN (  
    SELECT department_id, AVG(salary) AS avg_salary
```

```
FROM employees

GROUP BY department_id

) d

ON e.department_id = d.department_id

WHERE e.salary > d.avg_salary;
```

♦ Explanation:

Inline View (Subquery):

```
sql

(

    SELECT department_id, AVG(salary) AS avg_salary

    FROM employees

    GROUP BY department_id

)
```

- This subquery calculates the **average salary** for each department.
- It becomes a **temporary result set** (aliased as d) available to the outer query.

Outer Query:

- Joins the employees table with the inline view d.
 - Filters employees where their salary is **greater than the average** salary of their department.
-

✓ Result (Based on Sample Data):

EMPLOYEE_ID	FIRST_NAME	SALARY	AVG_SALARY
102	Alice	9000	8000
103	Bob	10000	10000

Only Alice is returned (John and Carol earn less than 8000, Bob earns equal to avg of 10000).

Scalar subquery:

In Oracle SQL, a scalar subquery is a subquery that returns exactly one value (a single row and one column) and can be used anywhere a single value (scalar) is allowed – such as in the SELECT, WHERE, or ORDER BY clauses.

♦ Characteristics of Scalar Subqueries

- Returns one row and one column (i.e., one value).

- Can be used **in place of a constant or column**.

If it returns more than one row → Oracle throws:

sql

ORA-01427: single-row subquery returns more than one row

♦ Example 1: Scalar Subquery in SELECT

sql

```
SELECT first_name,  
  
       salary,  
  
       (SELECT AVG(salary) FROM employees) AS avg_salary  
  
FROM employees;
```

✓ Explanation:

- The subquery (SELECT AVG(salary) FROM employees) returns a **single value** (e.g., 7800).
 - That value is shown alongside each employee's row.
-

♦ Example 2: Scalar Subquery in WHERE

sql

```
SELECT first_name, salary  
  
FROM employees  
  
WHERE salary > (  
  
    SELECT AVG(salary)  
  
    FROM employees  
  
);
```

✓ Explanation:

- Filters employees whose salary is **greater than the average**.
- The scalar subquery returns one value (e.g., 7800).

Introduction to constraints:

◆ Introduction to Constraints in Oracle SQL

In **Oracle SQL**, **constraints** are rules enforced on table columns to maintain **data integrity, consistency, and accuracy** in the database.

They ensure that only **valid data** is inserted, updated, or deleted.

◆ Why Use Constraints?

Constraints help:

- Prevent invalid or inconsistent data

- Enforce relationships between tables
 - Apply business rules at the database level
-

♦ Types of Constraints in Oracle SQL

Constraint Type	Description
PRIMARY KEY	Ensures each row has a unique and not null identifier.
FOREIGN KEY	Enforces a relationship between two tables.
UNIQUE	Ensures all values in a column (or group of columns) are unique .
NOT NULL	Ensures a column cannot have null values.
CHECK	Ensures values in a column meet a specific condition .

Not null constraints:

♦ NOT NULL Constraint in Oracle SQL

The **NOT NULL constraint** in Oracle SQL ensures that a **column must always have a value** – it **cannot be left empty** (NULL). This is the most basic data integrity rule.

♦ Purpose

- Prevents missing or incomplete data.
 - Enforces that a value **must be provided** during an INSERT or UPDATE.
-

♦ Syntax 1: Define NOT NULL When Creating a Table

sql

```
CREATE TABLE employees (  
  
    employee_id    NUMBER NOT NULL,  
  
    first_name     VARCHAR2(50) NOT NULL,  
  
    last_name      VARCHAR2(50),  
  
    hire_date      DATE NOT NULL  
  
);
```

✅ In this example:

- employee_id, first_name, and hire_date **must have values** when inserting data.

♦ Syntax 2: Add NOT NULL to an Existing Column

sql

```
ALTER TABLE employees
```

```
MODIFY last_name NOT NULL;
```

! You can only do this if the column **already** has no NULL values.

♦ Inserting Data – Example

sql

```
INSERT INTO employees (employee_id, first_name, hire_date)
```

```
VALUES (101, 'Alice', TO_DATE('2023-01-01', 'YYYY-MM-DD'));
```

✓ This will work because all NOT NULL columns are provided.

sql

```
INSERT INTO employees (employee_id, hire_date)
```

```
VALUES (102, TO_DATE('2023-01-01', 'YYYY-MM-DD'));
```

✗ This will fail because first_name is **NOT NULL** but was not provided.

♦ Checking for NOT NULL Constraints

sql

```
SELECT column_name, nullable
```

```
FROM user_tab_columns
```

```
WHERE table_name = 'EMPLOYEES';
```

- nullable = 'N' → the column has a NOT NULL constraint.